



ARTIFICIAL INTELLIGENCE SOFTWARE DEVELOPMENT

CST8510 Week 9
Dr. Hari M Koduvely



Agenda for Today

□ Theory: 6:00PM – 8:00PM

- Right infrastructure for ML Systems
- Four layers of infrastructure
- ML Resource Management
- ML Platform

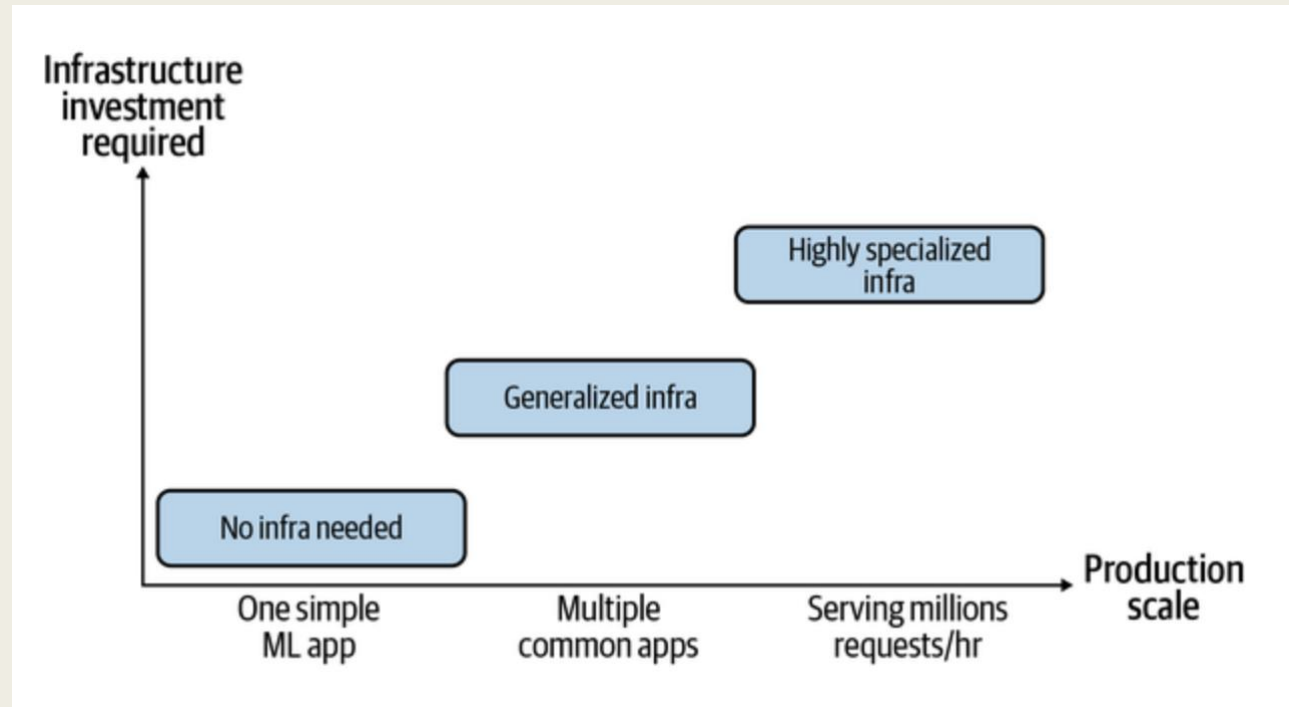
□ Lab: 8:00PM – 10:00PM

- Standup Meetings

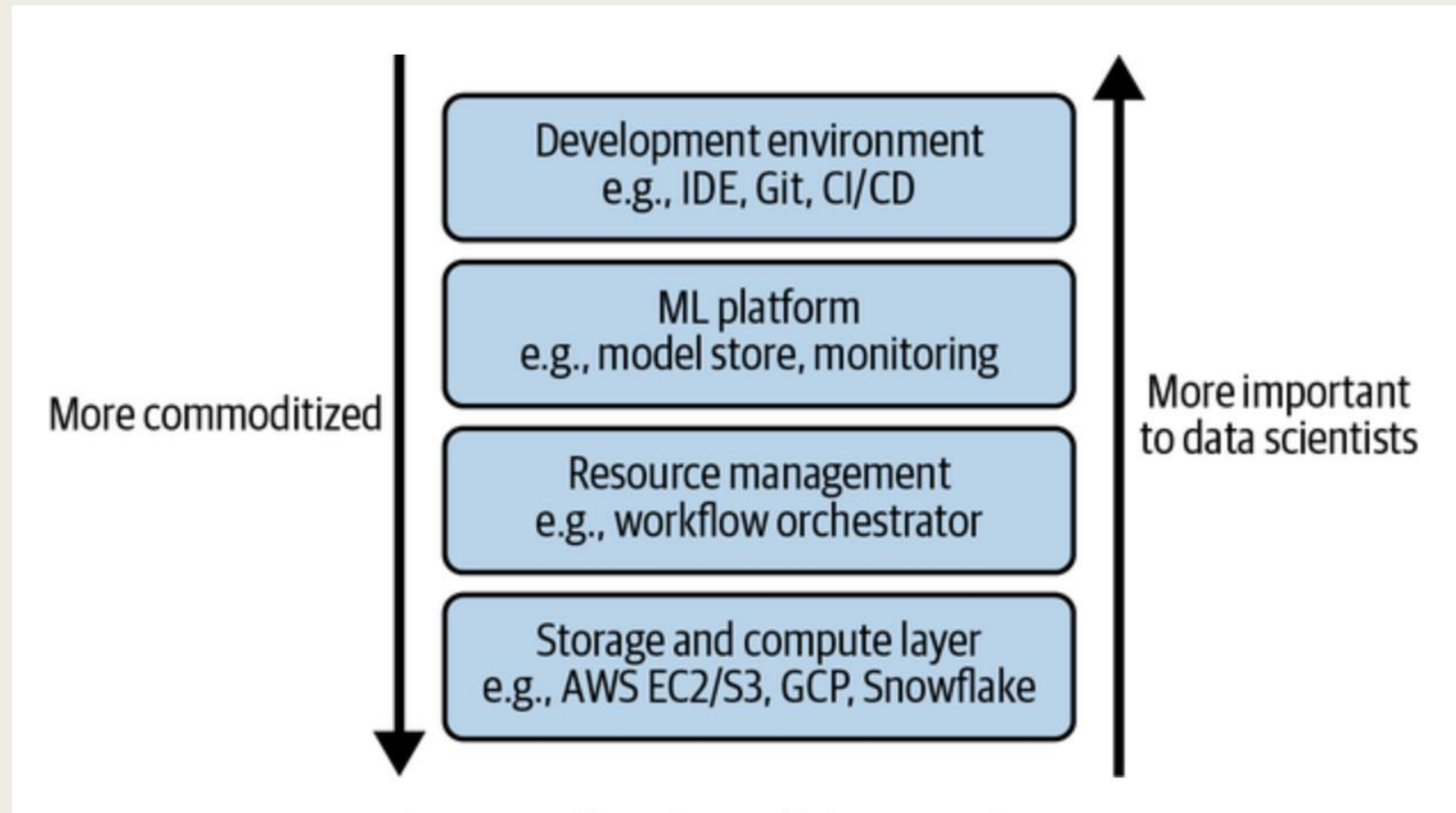


Infrastructure and Tooling for MLOps

- ❑ Infrastructure requirement of different companies.

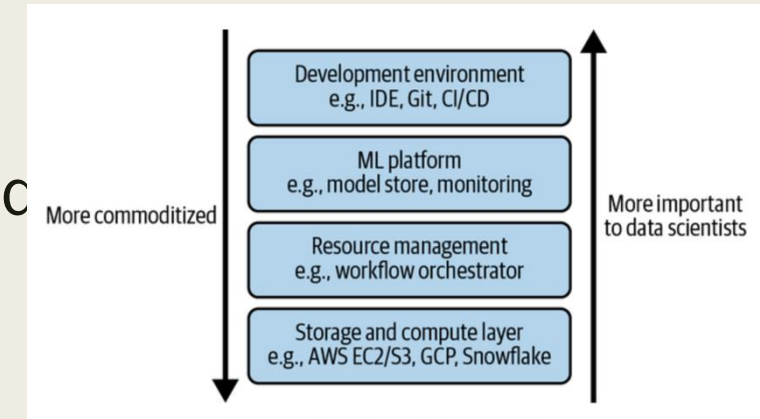


Four Layers of Infrastructure



Four Layers of Infrastructure

- ❑ Storage and Compute
 - Layer where data is collected and stored
 - ML workloads are run
- ❑ Resource Management
 - Schedule and orchestrate ML workloads
 - Airflow, Kubeflow etc.
- ❑ ML Platform
 - Model stores, feature stores and monitoring tools
 - SageMaker, MLFlow
- ❑ Development Environment
 - Layer where code is written and experiments are run



Storage and Compute Layer

- ❑ Storage layer is where data is stored and collected
- ❑ At the basic level storage can be on
 - Hard Drive Disk (HDD)
 - Solid State Drive (SSD)
- ❑ Storage layer is completely commoditized and moved to cloud

Storage and Compute Layer

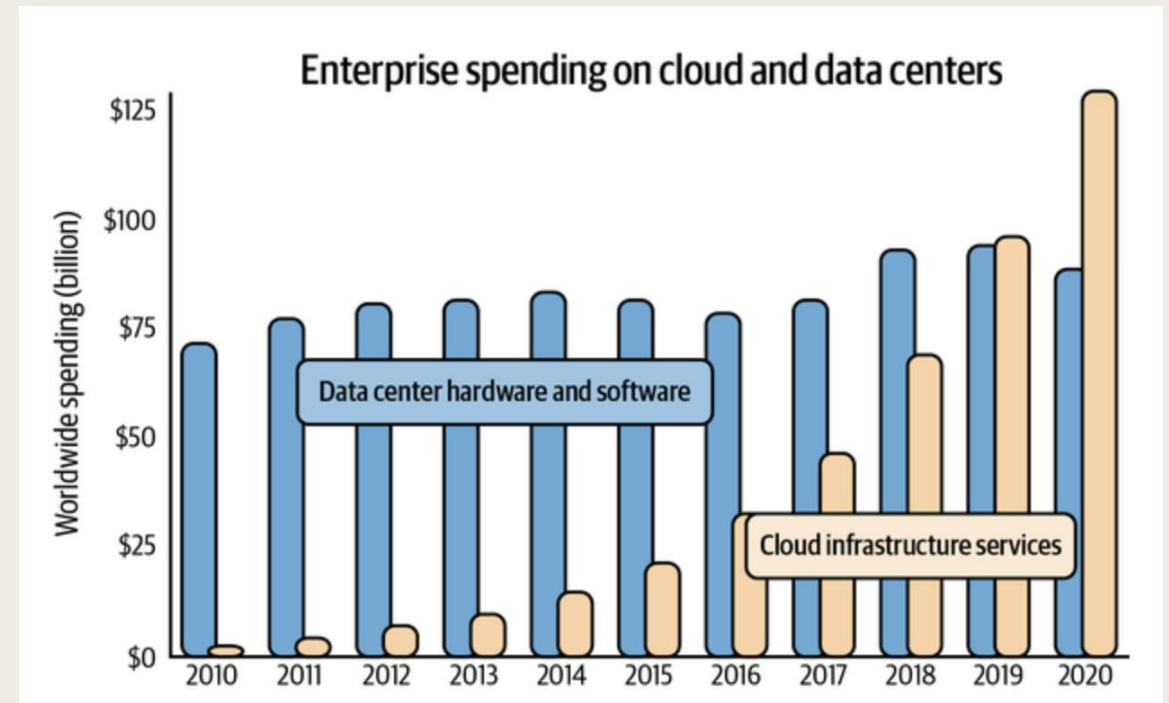
- ❑ Compute layer refers to all the compute resources available
- ❑ Amount of compute resources available determines the scalability of ML workloads
- ❑ The compute layer can usually be sliced into smaller compute units to be used concurrently
 - Threads
 - Containers
 - Pods

Storage and Compute Layer

- ❑ Compute units are characterized by two metrics.
 - How much memory it has (GB units)
 - How fast it runs an operation (FLOPS)
- ❑ Amount of compute resources available determines the scalability of ML workloads.
- ❑ Compute Utilization – Ratio of the number of FLOPS a job can run to the number of FLOPs a compute unit is capable of handling.
- ❑ Practically one can only achieve utilization ~ 50%

Storage and Compute Layer

- ❑ Cloud spending accounts for approximately 50% cost of revenue of public software companies (analysis by a16z capital venture)
- ❑ Some companies are doing "cloud repatriation"



Development Environment

- ❑ Environment is where:

- Code is written
- Experiments are conducted
- Interaction with production environment happens

- ❑ Dev environment consists of the following components:

- IDE (Jupyter Notebook, VS Code, Pycharm etc.)
- Versioning software (Git, DVC, Weights and Biases etc.)
- CI/CD (Jenkins etc.)

Notebooks

- ❑ Notebooks are more than just IDEs, one can include:
 - Images
 - Documentation in LaTeX
 - Other artifacts like tables
- ❑ Notebooks are Stateful
 - Retains state after run is completed
 - If program fails, one can restart from where it failed
 - Ideal for doing experiments with large datasets
- ❑ Order of execution of the cells is important to keep track

Notebooks

- ❑ Companies like Netflix used Notebooks in the production env
- ❑ Other tools are developed to run on top of Notebooks
 - **Papermill** - for spawning multiple notebooks with different parameter sets
 - **Commuter** - A notebook hub for viewing, finding, and sharing notebooks within an organization.
 - **nbdev** - a library to write documentation and tests in the same place

Containers

- ❑ Production workloads spread out on multiple instances.
- ❑ Number of instances dynamically changes upon demand for predictions.
- ❑ When a new instance is created, one needs to install dependencies using a list of predefined instructions.
- ❑ Container technology is used for this purpose

Docker Containers

- ❑ A lightweight, stand-alone, and executable software package.
- ❑ Includes everything needed to run a piece of software (code, runtime, system tools, libraries, and settings)
- ❑ Containers isolate software from its environment, ensuring that it works consistently across different systems.
- ❑ Docker containers are built from images.
- ❑ These are templates containing the application's code, runtime, and other dependencies.

Benefits of Docker Containers

- ❑ **Portability:** Containers can run on any system with Docker installed, regardless of the underlying infrastructure.
- ❑ **Consistency:** Containers ensure that applications behave the same way in development, testing, and production environments.
- ❑ **Isolation:** Containers run in their own environment, minimizing conflicts with other applications or system components.
- ❑ **Scalability:** Containers can be easily scaled up or down, making it simpler to manage application loads and resources.
- ❑ **Version control and sharing:** Docker images can be versioned and shared through repositories like Docker Hub, enabling collaboration and easy updates.

Example of a Docker Container

- ❑ Download the latest PyTorch base image.
- ❑ Clone NVIDIA's apex repository on GitHub, navigate to the newly created *apex* folder, and install apex.
- ❑ Set *fancy-nlp-project* to be the working directory.
- ❑ Clone Hugging Face's transformers repository on GitHub, navigate to the newly created *transformers* folder, and install transformers.

Example of a Docker Container

```
FROM pytorch/pytorch:latest
```

```
RUN git clone https://github.com/NVIDIA/apex
```

```
RUN cd apex && \
```

```
    python3 setup.py install && \
```

```
    pip install -v --no-cache-dir --global-option="--cpp_ext" \
```

```
    --global-option="--cuda_ext" ./
```

```
WORKDIR /fancy-nlp-project
```

```
RUN git clone https://github.com/huggingface/transformers.git && \
```

```
    cd transformers && \
```

```
    python3 -m pip install --no-cache-dir.
```

PODs

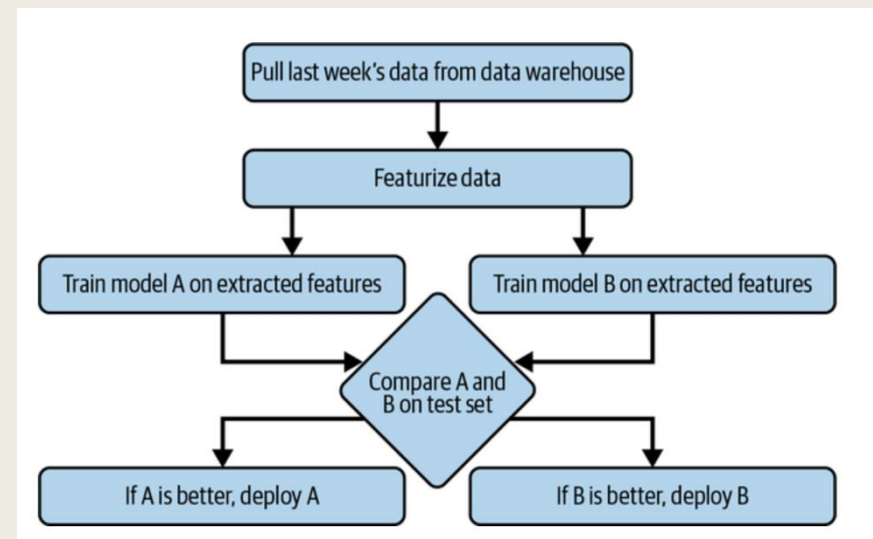
- **Basic Concept:** A pod is a group of one or more containers, with shared storage/network, and a specification for how to run the containers. It is the smallest deployable unit of computing that can be created and managed in Kubernetes.
- **Shared Context:** Containers in the same pod share the same IP address and port space, and can find each other via localhost. They can also share mounted storage.
- **Atomic Unit:** In Kubernetes, the pod is the atomic unit of scaling. When you scale an application up or down, you're actually increasing or decreasing the number of pods.
- **Use Case:** Pods are used when there is a need for a few containers to work together very closely as a single cohesive unit of service.

Resource Management

- ❑ In the pre-cloud world resources were limited.
- ❑ Focus was then on maximizing resource utilization.
- ❑ In the cloud world focus is on using resources cost-effectively.
- ❑ Two characteristics of ML workflows that influence their resource management.
 - Repetitiveness
 - Dependencies

Schedulers and Orchestrators

- ❑ Cron -.scheduling repetitive jobs to run at fixed times.
- ❑ Can not handle the dependencies between the jobs it runs.
- ❑ Schedulers are Cron programs that can handle dependencies.
- ❑ Takes in the DAG of a workflow and schedules each step accordingly.
- ❑ Tend to leverage queues to keep track of jobs
- ❑ Need to be aware of the resources available and the resources needed to run each job



Schedulers and Orchestrators

Example of scheduling a job using Slurm

```
#!/bin/bash
```

```
#SBATCH -J JobName
```

```
#SBATCH --time=11:00:00      # When to start the job
```

```
#SBATCH --mem-per-cpu=4096  # Memory, in MB, to be allocated per CPU
```

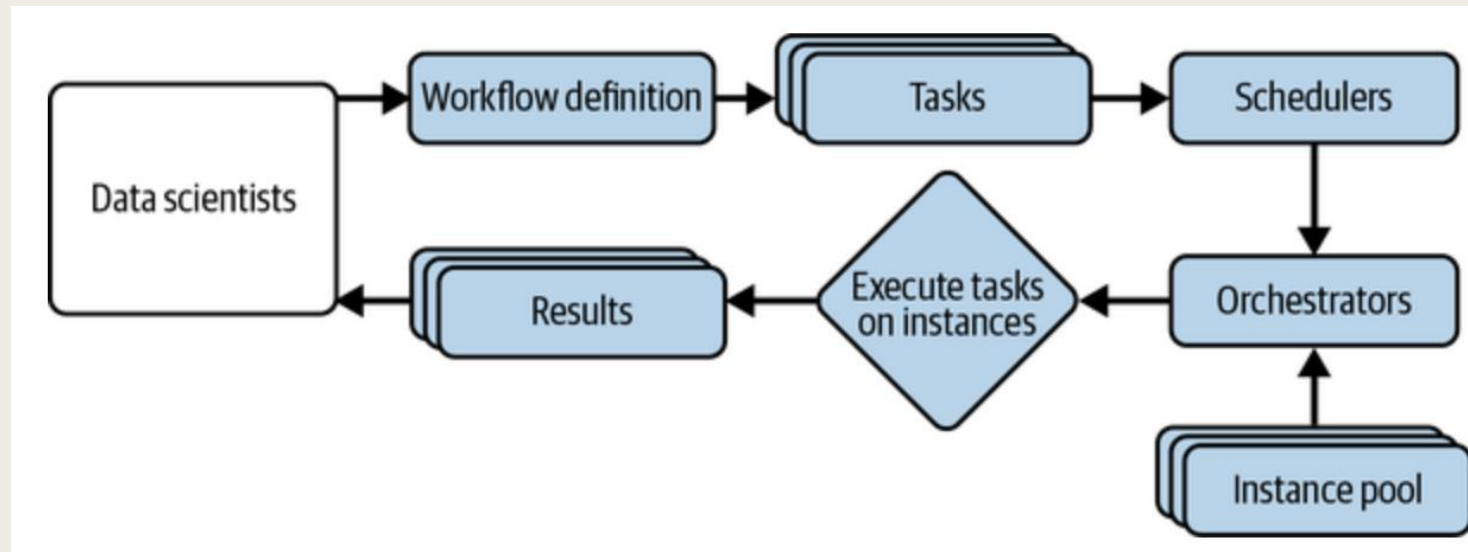
```
#SBATCH --cpus-per-task=4    # Number of cores per task
```

Schedulers and Orchestrators

- ❑ Schedulers are concerned with when to run jobs and what resources are needed to run those jobs.
- ❑ Orchestrators are concerned with where to get those resources.
- ❑ Schedulers deal with job-type abstractions such as DAGs, priority queues, user-level quotas.
- ❑ Orchestrators deal with lower-level abstractions like machines, instances, clusters, service-level grouping, replication, etc.
- ❑ It can dynamically increase/decrease the number of instances in the available instance pool.
- ❑ Most well-known orchestrator today is Kubernetes.

Workflow Management

- ❑ They allow you to specify your workflows as DAGs.
- ❑ Workflows can be defined using either code (Python) or configuration files (YAML).
- ❑ Once a workflow is defined, the underlying scheduler usually works with an orchestrator to allocate resources to run the workflow



Airflow

- ❑ One of the first workflow management tool.
- ❑ Developed at Airbnb and open sourced.
- ❑ Contains a huge library of operators.
- ❑ Easy to use with different cloud providers, databases, storage options.

Airflow Drawbacks

- ❑ Airflow is monolithic - it packages the entire workflow into one container.
- ❑ Airflow's DAGs are not parameterized - you can't pass parameters into your workflows.
- ❑ If you want to run the same model with different learning rates, you'll have to create different workflows!
- ❑ Airflow's DAGs are static - it can't automatically create new steps at runtime as needed.
- ❑ Next generation of workflow orchestrators **Argo** and **Prefect** address these issues.

ML Platform

- ❑ ML Platform is a relatively new concept like MLOps
- ❑ No universal standard definition exists.
- ❑ The shared set of tools for ML deployment makes up the ML platform.
- ❑ Most important components:
 - Model Deployment
 - Model Store
 - Feature

ML Platform

- ❑ Two important criteria for evaluating the component tools:
 - Whether the tool works with your cloud provider or allows you to use it on your own data center.
 - Need to run and serve models from a compute layer, and usually tools only support integration with a handful of cloud providers.
- ❑ Whether it is an open source or a managed service:
 - Opensource tools can be hosted by engineers
 - Less about data security and privacy.
 - More Eng resources are required.
 - Managed services could be more expensive.
 - May not comply with regulations of data storage and privacy.

ML Platform – Model Deployment

- ❑ A deployment service helps in both pushing models and their dependencies to production and exposing them as endpoints.
- ❑ Deployment is the most mature among all ML platform components.
- ❑ All major cloud providers offer tools for deployment:
 - AWS – SageMaker
 - Azure – AzureML
 - GCP – VerTexAI
 - Startups:
 - MLFlow Models:
 - Seldon
 - Cortex
 - Ray-Serve

ML Platform – Model Store

- ❑ To help with debugging and maintenance, it is not sufficient to store model object alone
- ❑ Information about models stored:
 - Model Definition: Loss function, number of layers of NN, number of parameters in each layer
 - Model Parameters: Actual values of the model parameters after training.
 - Featurize and Predict functions
 - Dependencies: Python packages
 - Data: Pointers to data storage
 - Model Generation Code: Pointers to Github repo
 - Experiment artifacts: Loss curves, performance metrics
 - Tags

ML Platform – Feature Store

❑ Why Feature store is needed ?

- Feature management
- Feature computation
- Feature consistency

❑ Popular tools for feature store:

- Feast:
 - Strong in creating batch features
 - Weak in creating streaming features
- Tecton:
 - Capable of storing both online and batch features
 - Require deep integration

Summary of Today's Learning

- ❑ Different layers of ML infrastructure.
- ❑ Scheduling and orchestration of different ML tasks.
- ❑ ML resource management
- ❑ Important components of ML platform

Thank You